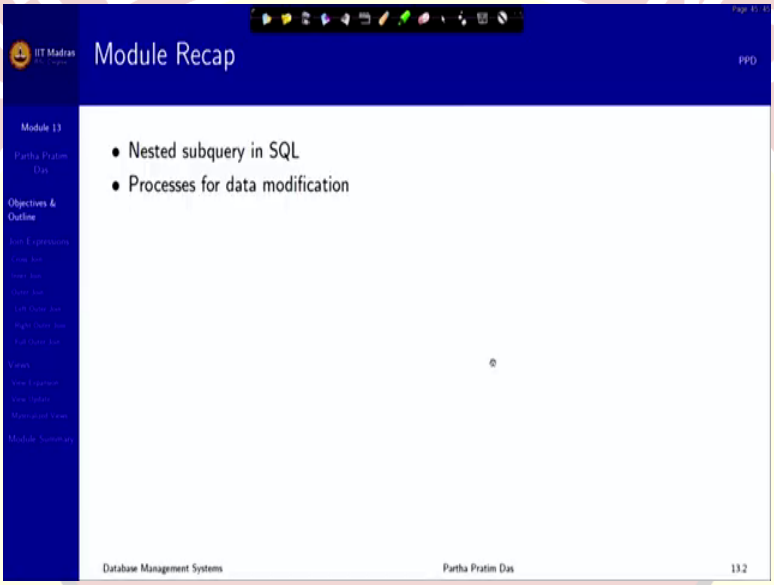
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Intermediate SQL/2

Welcome to Module 13 of Database Management Systems course in the IIT Madras online B.Sc. program.

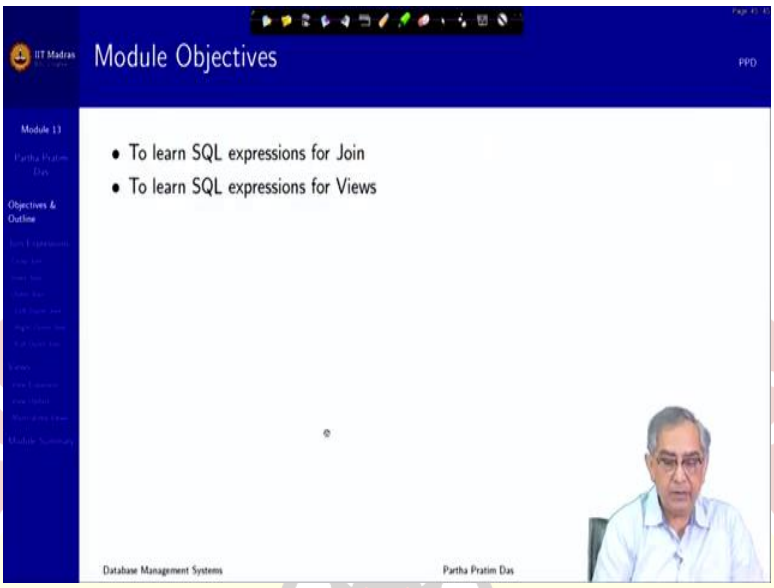
(Refer Slide Time: 0:25)



We have been discussing on intermediate level features of SQL language, in the last module we discussed about two very important or group of features, one is in terms of nested sub query where a query output the sub query output can be used as a part of the from clause or in the predicate in where clause or even on the select clause, which makes writing of several queries really simpler, conceptually simpler, semantically clear.

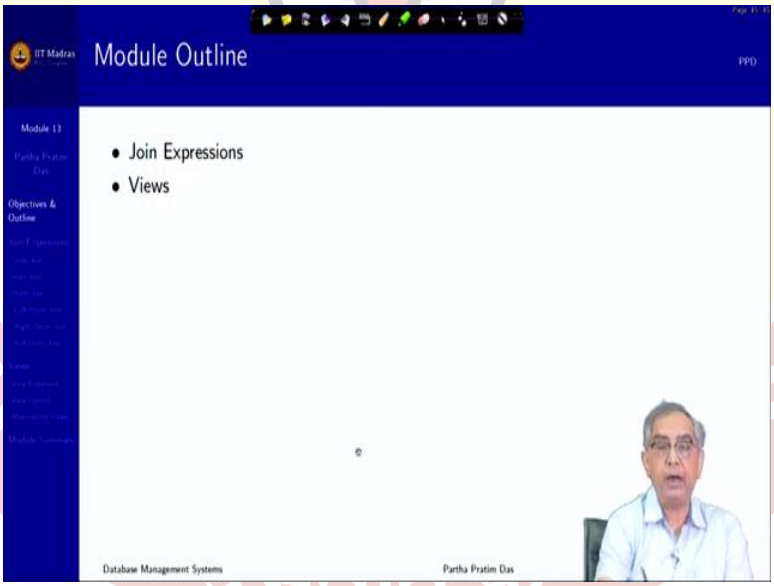
And we also talked about the basic data modification features of SQL in terms of delete, insert and update.

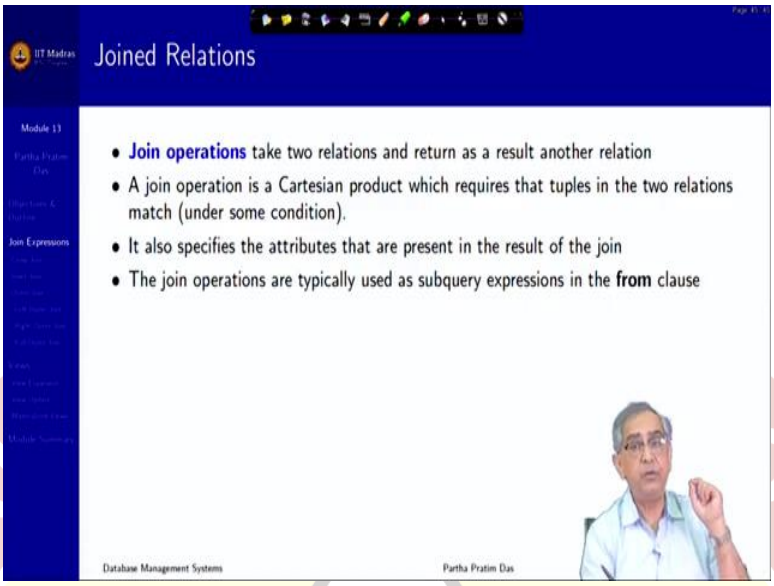
(Refer Slide Time: 1:13)



We will continue on the intermediate features, we will talk about two more very important features of SQL, one is on join, other is on views.

(Refer Slide Time: 1:25)

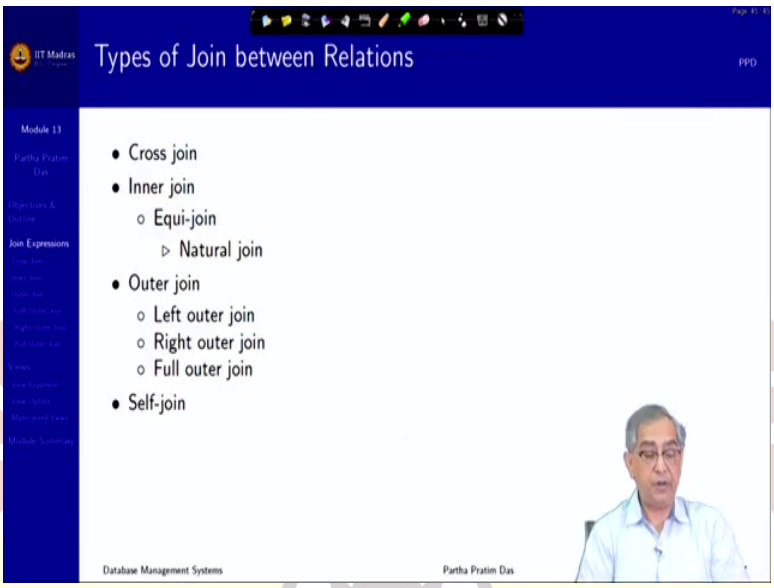




We have been talking about join being done in terms of doing it kind of in the form of select from where clause by which it can also be done. Well often it becomes easy, if we can use explicit join expressions and connect the two relations to create a new result relation. As we have seen join operation is basically a Cartesian product, but it has certain conditions on the Cartesian product which the tuples of the two relations have to match.

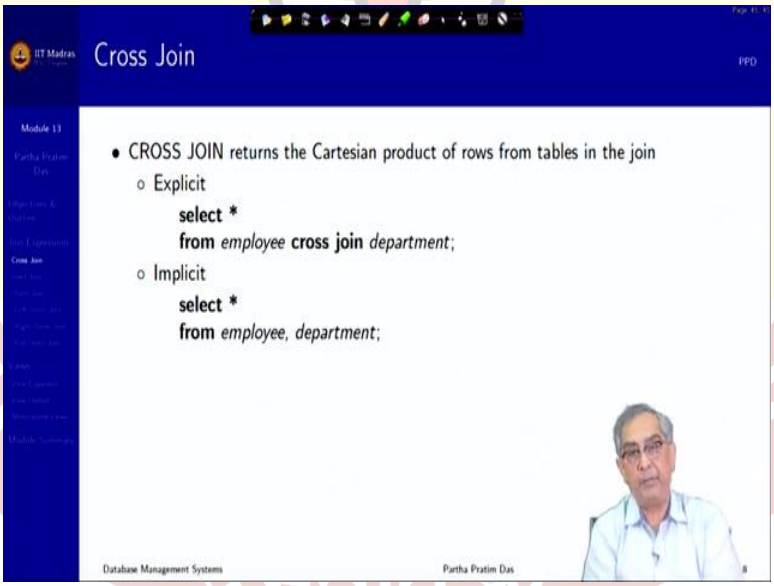
And depending on that condition, depending on what we want to do in that condition, we can, we have different classes of join that happens and join can also like select, join can also specify the attributes that will be present in the result. And it is not always but in several cases join expressions are actually used as a sub query in the from clause so that you have a new, you had your two relations, you have joined them, you have interrelationships between different attributes of the same table which belong to two different tables now, and you use that to do further computation is a more significant use model for join we will see those more later.

(Refer Slide Time: 3:00)



So, these are different types of join, I will not go through these explaining we will take example of each one as we go along and to be clear.

(Refer Slide Time: 3:11)

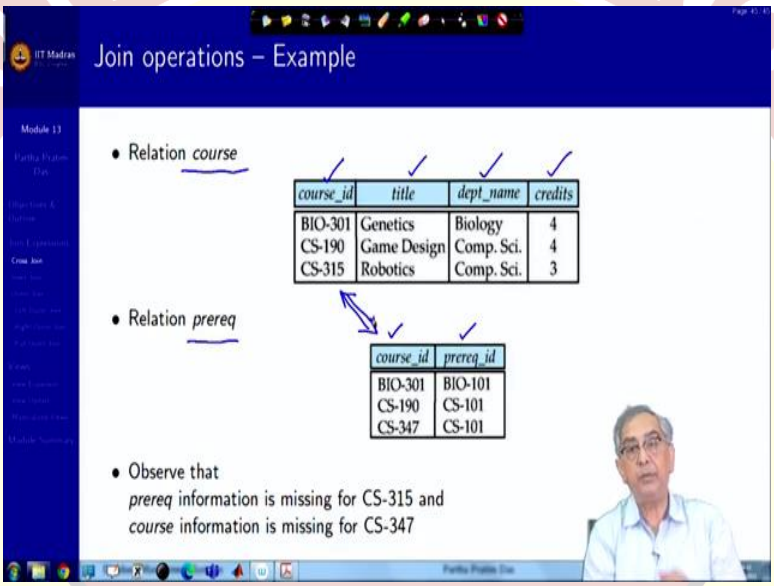


Cross join is they are both for completeness, it is a simple Cartesian product of rows from tables in the join. So, we say select * which means all attributes of first table employees and the second table department and how do you get this table by doing a cross join which means take every tuple of employees, take every table of department, match them in all possible ways if you have

m employees tuple and n department tuple, if you match them it is m times n number of total tuples with total set of attributes is what you get.

And you have this is an explicit way of doing it given as I said more for completeness and semantic compliance I should say. But we have seen that implicitly the same result will come if I do select * from employee comma department. Well, so cross join is probably not that interesting, what would interest us is about to come.

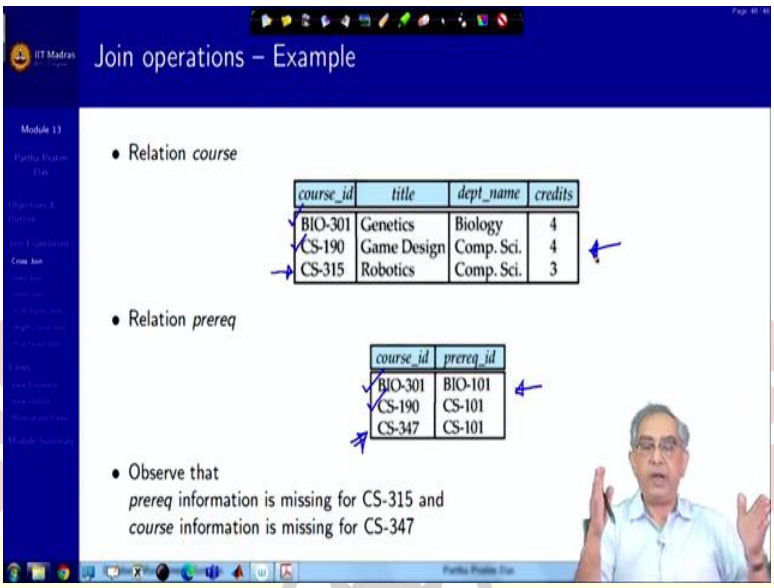
(Refer Slide Time: 4:20)



Now, I start with a very, very tiny example to be able to illustrate all different types of joins that we will talk about. So we are, we will be using two relations course and prerequisite of the course. So, course as you see have course ID which is the key. You have the title, department name, credit, these are the different attributes.

In prerequisite you have course ID and prerequisite ID, again the course ID is the key. So, course ID is common between these two. So, Course table tells you about everything, every other property of courses, but prereq table or this relation tells you about a prerequisite of this.

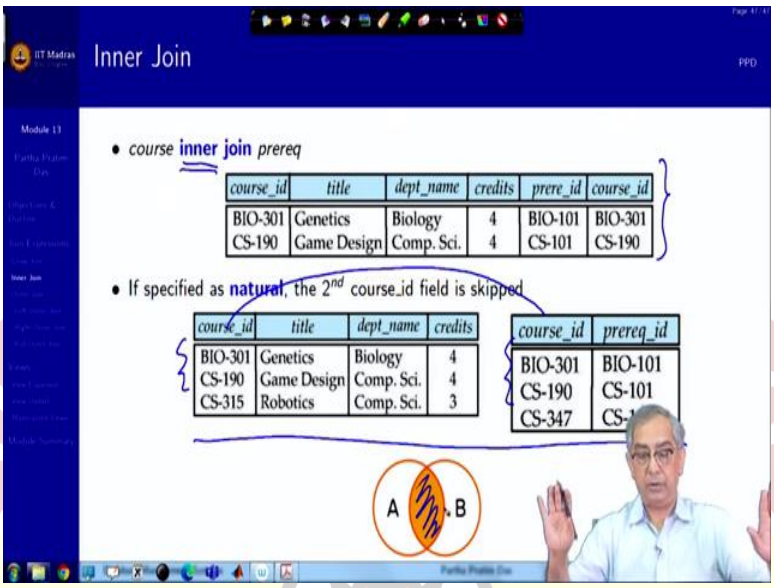
(Refer Slide Time: 5:13)



Look at the specific choice of example, if you look at BIO-301 here, it has a prerequisite given, if you have CS-190 it has a prerequisite specified, but if you look at CS-315 there is no entry here in the prereq table. So, the prerequisite of CS-315 is not known. Similarly, if you look at the prerequisite table CS-347 you know the prerequisite, but that particular course does not exist in the courses table.

So, this is the kind of information gap that often will exist in reality. So, the, we would like to make a join between course and prerequisite in the presence of such incompleteness of information and see what are the different semantic dimensions in which the operations can be defined.

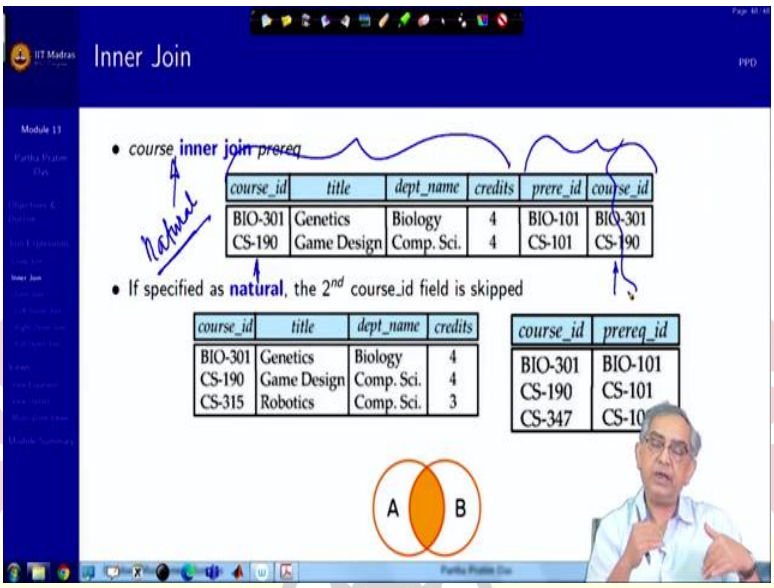
(Refer Slide Time: 6:18)



So, the first that we see is known as inner join, inner join is a join which will again certainly the same tables with the same set of tuples will be used all through the join is to make things clear, so this is what you have. Inner join is where you do the join naturally on course id as you can understand. And when they are equal you retain those tuple, if they are not required you throw it away.

And for missing tuples as we saw like CS-315 does not have a prerequisite and CS-347 does not have the course details, they are simply ignored. So, you have here in the inner join, mind you the word inner, in the inner join you have just the courses BIO-301 and CS-190 because they are in both. So, this is inner join. So, if you look at the two sets of course IDs, inner join is the intersection part, something that exists clearly in both.

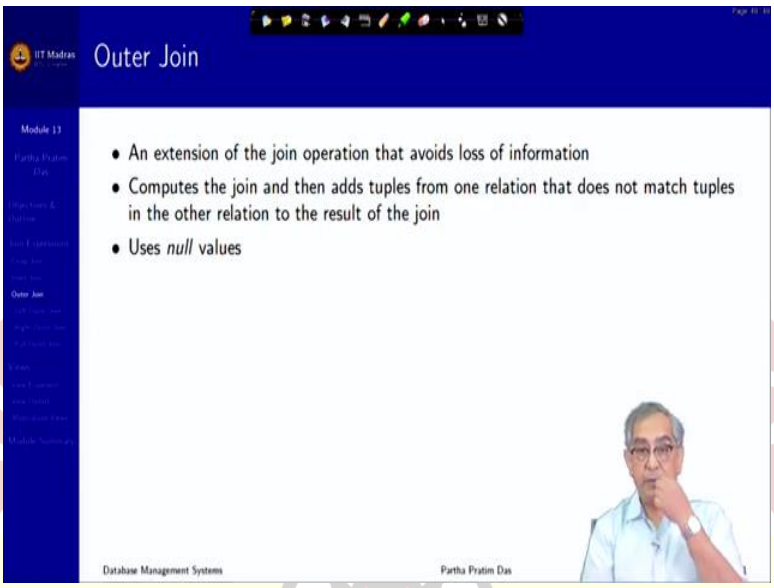
(Refer Slide Time: 7:32)



Now, one more thing is we could actually put another specified natural here, if we put to the specified natural, here you see that it is a join. So, you have all the attributes of the course table as well as you have all the attributes of the prereq table, everything is there *, 4 6, 4 plus 2 6 attributes, but if is a natural, then the attribute or attributes on which the join has been done, the equality has been done that will be skipped.

So, if I had said natural, then the result will be same only thing this column course ID and this column course ID which must be the same, one of them will be removed. So, everywhere you will find that when we do join if we specify natural, then the duplicity of the common column will be removed from the result, if we do not say natural that duplicity will be maintained. So, this is the most conservative form of doing join in the absence of some information.

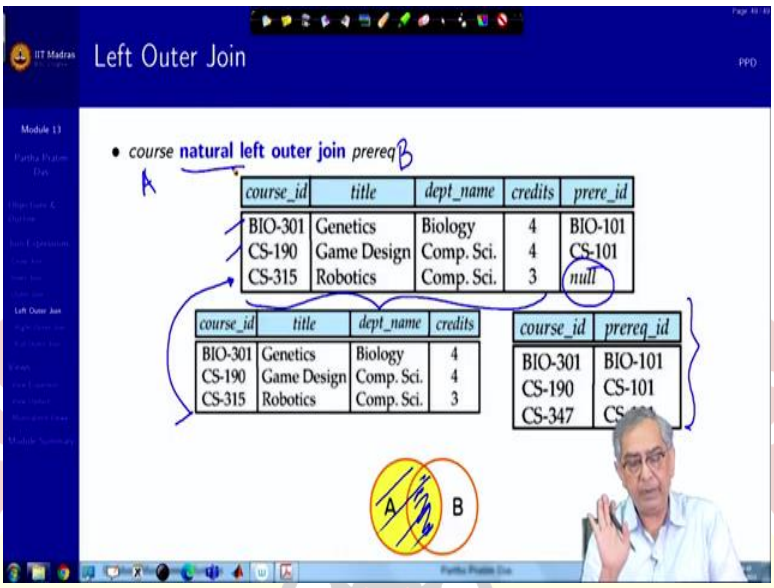
(Refer Slide Time: 8:41).

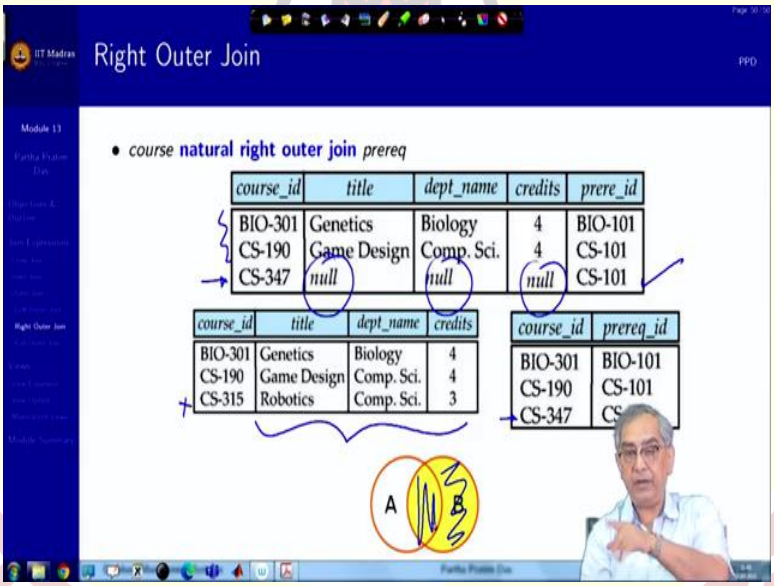


Now, but obviously, we are losing information in the result we do not have some courses which were there we do not have some courses for which prerequisite was known, what if we want to avoid that loss of information and say okay, we would maybe everything is not known, but still we would not like to lose the fact that CS-315 was a, is a course, just because we do not know the prerequisite we are throwing away the course.

Or just because we do not have other details like title and all we are training at the prerequisite requirement of sphere 347 is not what we want to do, rather we can use null values for things that we do not know. So, when we do this when you, so now we are trying to get more inclusive, so you are getting bigger. So, we said that is the notion of outer join.

(Refer Slide Time: 9:37)





So, outer join can be of two types, one is left and other is right. So, as you can see that this was the inner, this was the inner, the common part and this is A B other left and right sets or relations. So this is, this kind of is A and this kind of is B. So, what you doing the left outer is you make sure that all of A is included in the join, if it is not there correspondingly in B, then the B's attribute will get a null value.

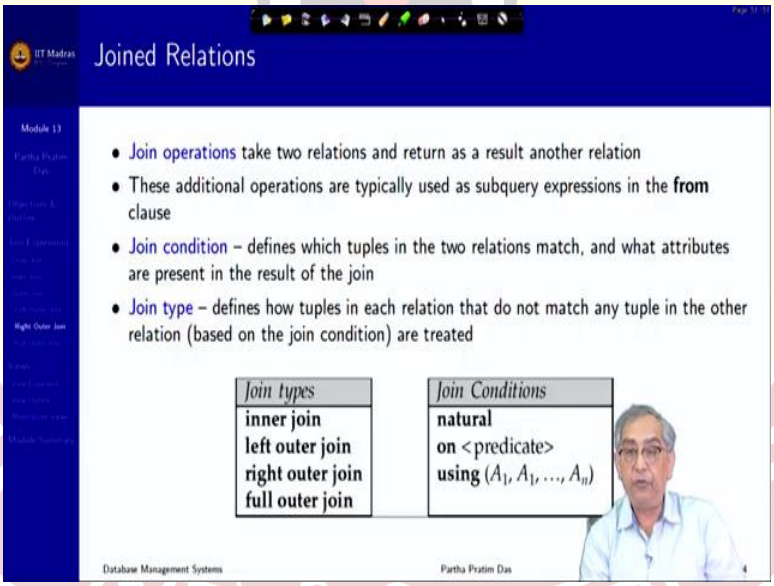
So, what we do we have the two tuples from the inner join as we had, CS-315 we do not lose, we put it here. So, it is all the same as it should be, it is does not have an entry in prereq, prereq there

is no entry. So, in the prereq ID we put a null, obviously, it must be nullable otherwise it will not work.

So, by that we make sure that all tuples which are there in the left relation will be there in the result. And then we have called it natural because you have called it natural the second course ID column has not come, if we do not call it natural the second course ID column will come, that I have already told you. So, it is clear that if we have a left outer join, I will have a way to preserve information for the second relation also or the right relation also. So, there is a right outer join.

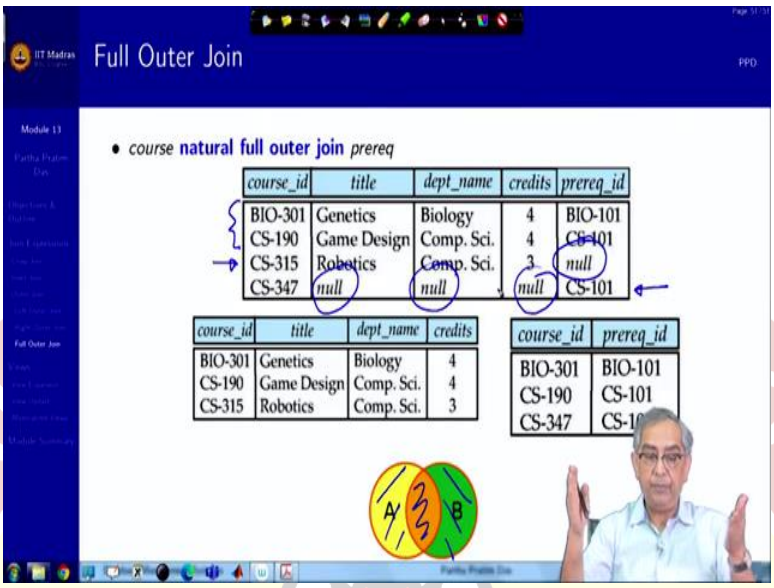
So, if I have right outer joins, these are same 347 is now retained and I do not know the information of these for 347. So, we have null, null, null, but we have the prerequisite information, but since I have done right outer join, we can see here whole of B is included in addition to the intersection. So, CS-315 does not feature here, it is natural therefore, the second course ID column is not there.

(Refer Slide Time: 11:47)



And since we have left and right, I can actually choose to have both of them. So, we have and when we do that, we call it a full outer join. So, inner join, left outer join, right outer join, full outer join and we have conditions like natural or on a particular predicate or using a certain attribute as to I may not always just go by name, I can say that I want to join of attribute x of relation r1 and attribute y of relation r2, this kind of things or a certain specific on a subset of attributes only.

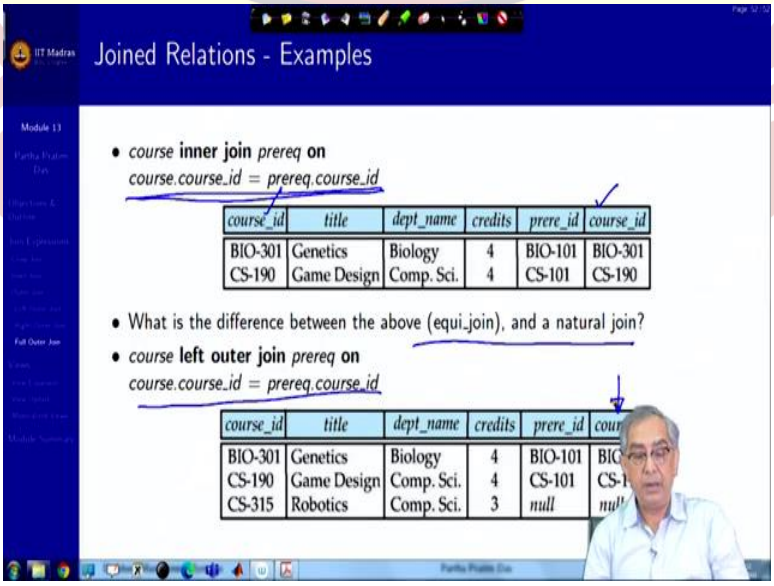
(Refer Slide Time: 12:23)



So, this is the example of full outer join. So, we have the inner join part, this is what we have from the left outer join, this is what we have from the right outer join. So, you can see that inner is, so, all of A is also included, all of B is also included, for the left we have a null here, for the right we have three nulls here. So, all information preserved you have now have a full picture.

Now, depending on the situation and the context and the semantics required you will have to decide which type of join you want to go for. Again put a natural otherwise the second course ID field will come.

(Refer Slide Time: 13:05)

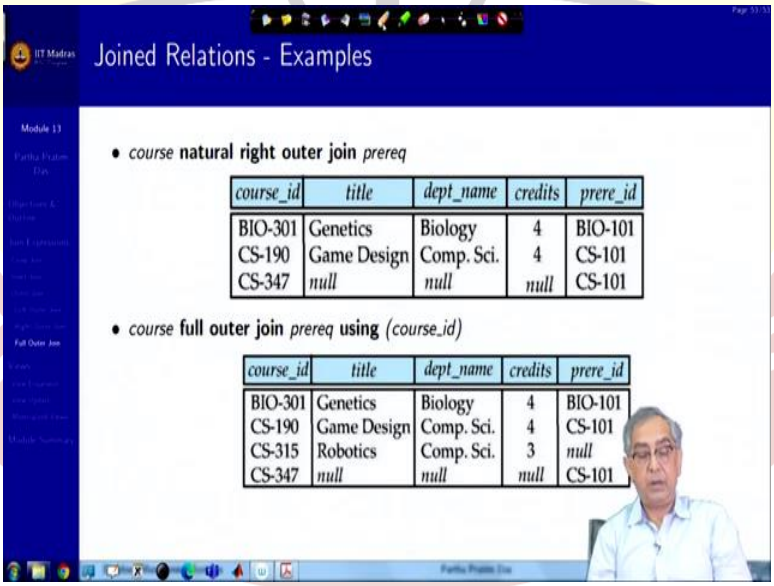


So, some examples, course inner join prereq on, prereq on is saying what are you joining on, so this is a join condition. Without that also it will join on this because the names are same, but just wanted to show you that you can put some condition, you can put some additional condition and or etcetera is a predicate.

So, you can actually make I mean we will talk specifically about that when we talk about relational algebra more we will say that as we have join, as we have natural join we have something called the theta join also, a theta join is a join with additional condition, I can say that you do a join, do a natural join, but make sure that only keep courses which are from biology department, something like that you can say that.

So, this is the, this is a condition this is on the predicate base join, again this is just a join not natural. Therefore, you see two course IDs. So, you can see the, what is the difference between this and a natural join, natural join you will not have the second course ID column. Similarly, we have a left outer join on this similar to the earlier result because it is not naturally not specified the second one comes but just shows you how to write predicates along with the join.

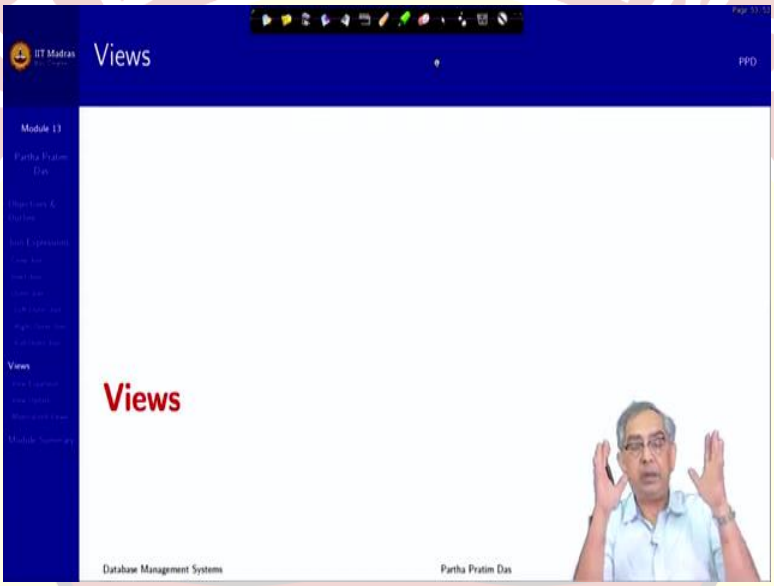
(Refer Slide Time: 14:33)



This is another exam two course natural right outer join prerequisite. So, since it is natural is written, the second course ID column is not there. Then you are saying that the full outer join prereq using course ID. So, this means that course ID any attribute that you put here must be there at both the relations otherwise, this will not make sense.

So, course ID is a common ID there maybe it is possible that there are other common IDs also, other common attributes also, but you do not want them to participate in the join, you want this particular course ID to participate. So, you are using course ID, which is common between course and prereq do a full outer join. So, this is that use of, so, you can do without any condition, you can have predicate condition, you can have using and so on. And this is the kind of result that you will get.

(Refer Slide Time: 15:34)



So, that is a kind of very important thing about how you because join is important because this is the only way very explicitly, semantically very explicitly, you can link information between two tables, two or more tables. At the same time, it has deep consequences, because as you can see that the basic process of join is making a Cartesian product.

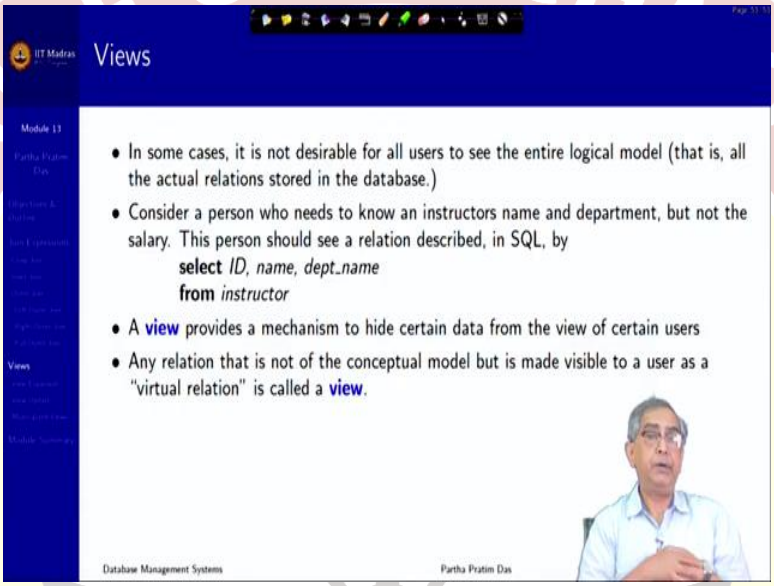
So, when you have 10 column, 10 rows in one table and 20 rows in other table, it is competing on 200, 10 into 20, 200 records and then throwing the rest away. But when you have say 1 million record here, and 1 million record in the other, 10 to the power 6 into 10 to the power 6 is 10 to the power 12, that is one tera number of records.

But finally, possibly you will have only maybe 10,000 records. But this intermediate cross product will kill you. So, join is a very interesting domain of query management because in terms of semantic expression, join is very expressive, very powerful, easy to write query, but join

is very expensive. So, there has to be smarter ways of making sure that we get the same result by doing less work than what we are just explicitly saying in the join.

So, we will come back to those aspects, I will be using join more semantically using variants to minimize the overhead looking into what could be alternate ways of writing the same query so that your intermediate table does not blow up, and so on, so forth.

(Refer Slide Time: 17:26)



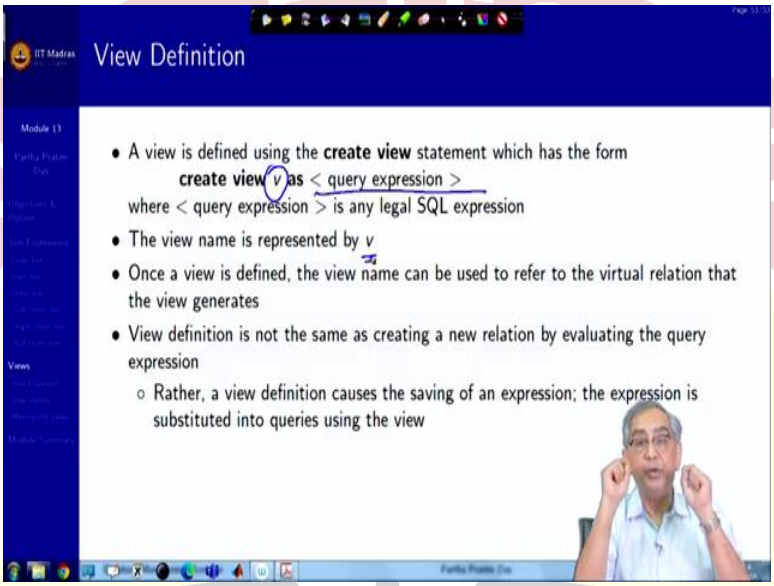
We will come to all those but let us pick up a second very, very interesting concept called view. If I may remind you at the very beginning, I said that there are three layers in terms of data abstraction, one is a physical, where bits and bytes we have not started talking about it, we will talk about it in later series of modules. Then you have the logical view, the conceptual view, this is what we have been dealing with all this time so far.

And when we said on top, we have multiple views, view level, which is actually not data as it is stored, but it is data as it is viewed or perceived. Because you do not want often to show the entire table to any XYZ user. You want to show, you do not want to expose your logical model, you do not want to give access to all kinds of information somebody dealing with.

Most of the people dealing with Course Management has to deal with instructors, departments, students and so on. But you do not want to expose the salary of the instructor or maybe even the budget of the department in most of these users. So, what you do? You create a view much in the same way you do the select statement.

So, you as a select statement gets you a projected, selected part of a table, what is a select statement, it is you have a full table, a select statement is taking certain columns and certain rows giving you another table. So, in the same way you can create a view with the exception that if view is a virtual relation.

(Refer Slide Time: 19:15)

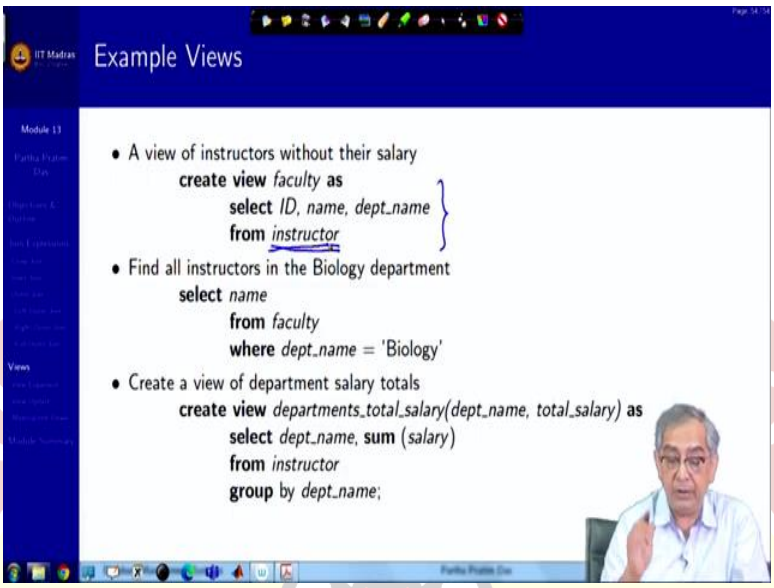


Let me explain what it means. So, you say that create view this is how you go about doing it you say create view as this is the view name as a query expression. So, it could be a select expression. So, this is the new. Now, the difference is when you do create table, a table physically gets created then insert, update, delete, records in that whatever you record is persisted stays there till you have made changes.

View is not that, view is more like an expression. So, what happens every time we use the view at that point of time the query expression you have used to define the view, create the view gets evaluated. And the then value of the relation that results from this query comes into your view.

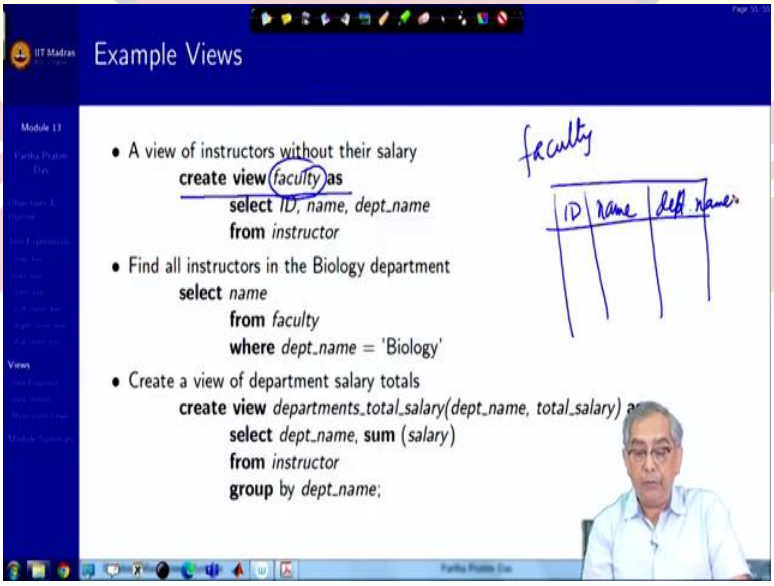
So, view is the dynamic thing, which is not getting stored in your database. But that is a feeling that anybody who is using it feels as if the relation is there. But the relation is not physically there. It is there through computational means. So, let us try to look at some specific examples.

(Refer Slide Time: 20:42)



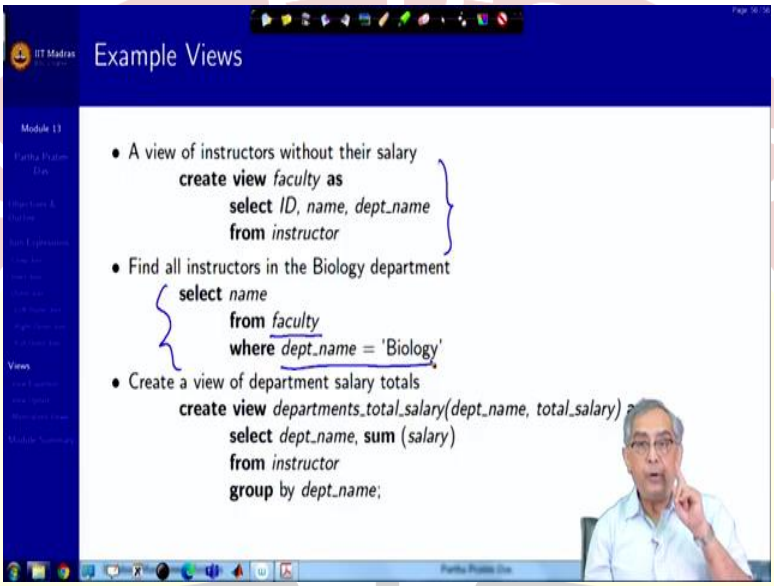
Let us talk about some specific examples. So, we are talking about the instructors. And we want to hide the salary. So what we do is, we create this query, these are query expressions, select ID, instructors ID, the name, the department name, which is normally for academic activities, this is what you need, these three. And you are saying from instructor, this is the original table, this is a persistent table. This can also be a view, we will come to that, but this is for now, this is original table.

(Refer Slide Time: 21:18)



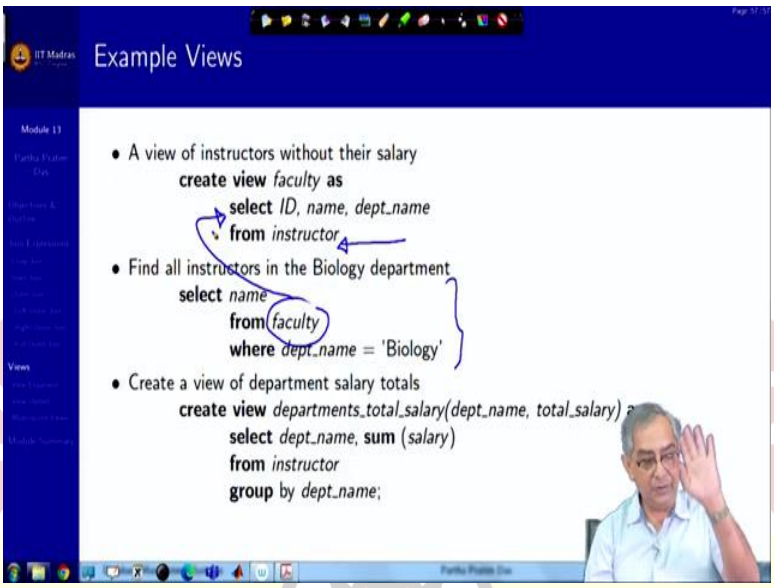
And then we say that create view as faculty, so the view created is faculty. So, faculty now is now like a table, so it has ID, it has name, it has depth name. And it is not only that, the view does not have the salary column or salary values, it does not even know that such a field exist, because it is hidden behind the query expression.

(Refer Slide Time: 22:01)



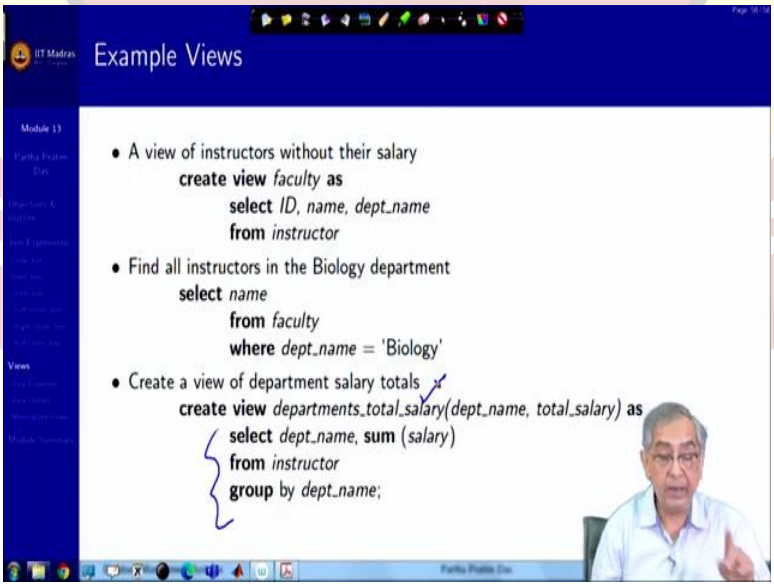
Now, when you, how do you use it? Use it simply as any other relation most often. So what you do, now we have a query using the view. So select name, we want to find the instructors of biology department. So, this is your new table of instructors called the faculty view. So, you are using that from faculty, take the name with the condition department is biology. So, even without knowing, even without knowing that an instructor relation exists, you are being able to take information out of it.

(Refer Slide Time: 22:45)



Now, the points always is since faculty is a virtual relation, it is not a persistent data. So, there are continuous changes happening in instructor. So, whenever you do this, you do not have a persistent faculty data, whenever you do this, it actually fires this query the nested form, so it is like a nested query happening, creates the faculty relation and then runs your outer query. That is how the view works.

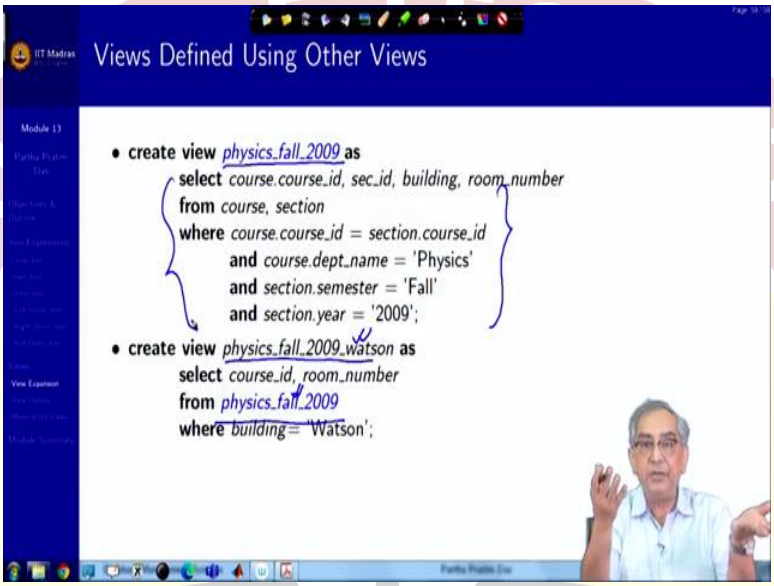
(Refer Slide Time: 23:12)



So, if you just conceptualize this rest of the view is just extension of common sense. So, you could create view which are, which have computed information also like here we are showing

that a view which has department total salary as a new view virtual relation of having department name and total salary, there is no physical relation which has that, but we are through this computation we are creating it. So, whenever we will use this, this nested query with the average function will execute and get me the tuples in the view.

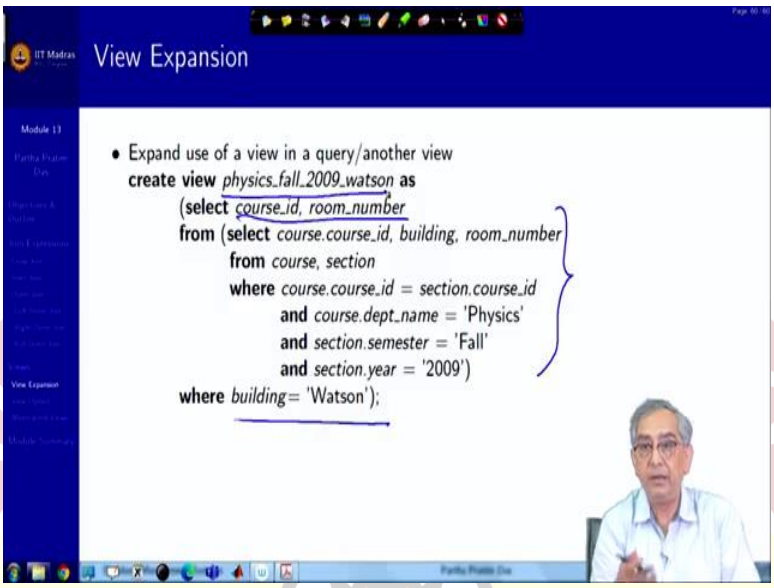
(Refer Slide Time: 23:55)



So, as we have seen, we can use a view into using it to solve queries. So, naturally, we can use it to create other views also. So, this is a view of physics_fall_2009, all courses offered in physics department in fall 2009. This will be easy for you to understand now, we have been doing it for long.

And then we are using another, we are creating another view by using this view. So, when we use this view, it will try to instantiate this view, when you try to instantiate this view, it will have to do this. So, it will keep on going like this till it actually gets the physical relations, computes all of them and then gives you the view result. So, that is the dynamic behavior of the view that you have.

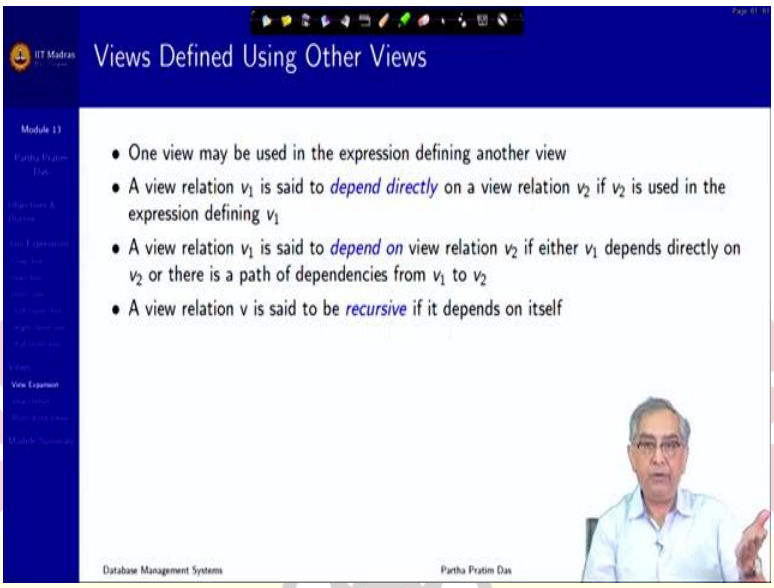
(Refer Slide Time: 24:51)



So, this kind of is called view expansion. That is you can have a view, build a view on top of that, build a view on top of that. Mix and match with any actual relation at any point of time without thinking about what is physically a relation and what is virtually a view relation. So, here you have one nested query on the from and then using that you have outer query with sets the Watson.

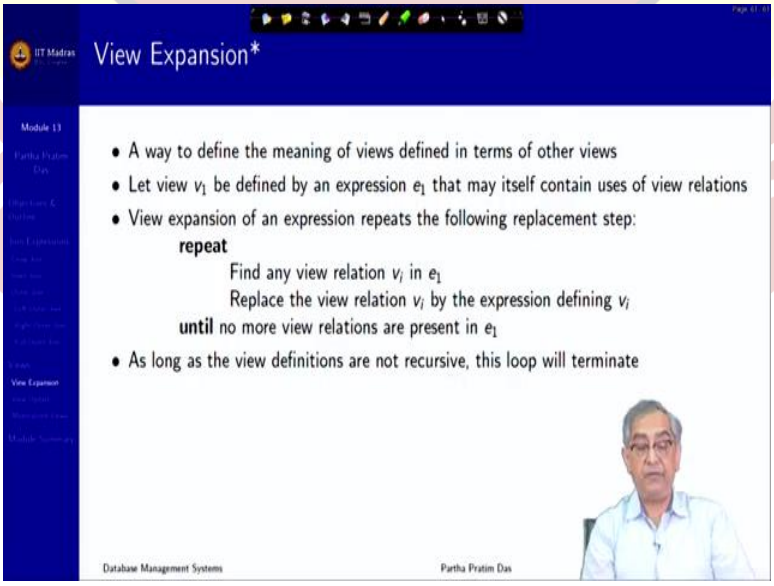
So, this is what you will actually get, when you do that definition of physics_fall_2009_Watson view using the physics_fall_2009 view. So, internally the system will expand and actually create this view and then evaluate it at the time when it is required.

(Refer Slide Time: 25:47)



So, this view expansion will use one view can use another view. So, a view relation v1 is said to depend directly on a view relation v2, if v2 is used in defining v1. As we have seen, v2 is used like physics_fall_2009 is used in defining physics_fall_2009_Watson. So, there is a direct dependence or the dependency could be transitive, that is v1 depends on v3, v3 depends on v4, v4 depends on v2. So, this view is depend on. So, all of these are fine as long as a relation, a view does not depend on itself, which is called recursive view.

(Refer Slide Time: 26:41)

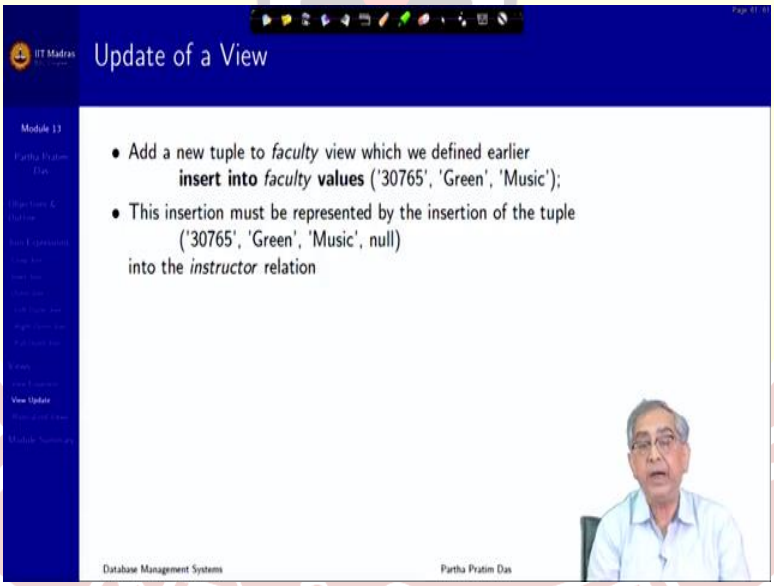


So, view expansion typically will have this kind of a for loop, find any view on relation in the expression, replace that view by the expression defining it as we just saw in physics_fall_2009 and physics_fall_2009_Watson, if that has further views keep on doing it till you are not left with any view.

Obviously, if the view definitions are not recursive, but this loop will necessarily terminate, does not mean that you cannot have recursive views. It is possible recursive view is a deep matter of deep subject in database in SQL. But we are not going to discuss recursive views in this course, because it is a little bit more advanced and it needs certainly background of recursive function theory which you are not prepared with.

But not all recursive views are useless. There are some very nice recursive views that are possible, but we are skipping those from the view expansion. That is why I put a * on the top of the slide saying that deeper of this is not done.

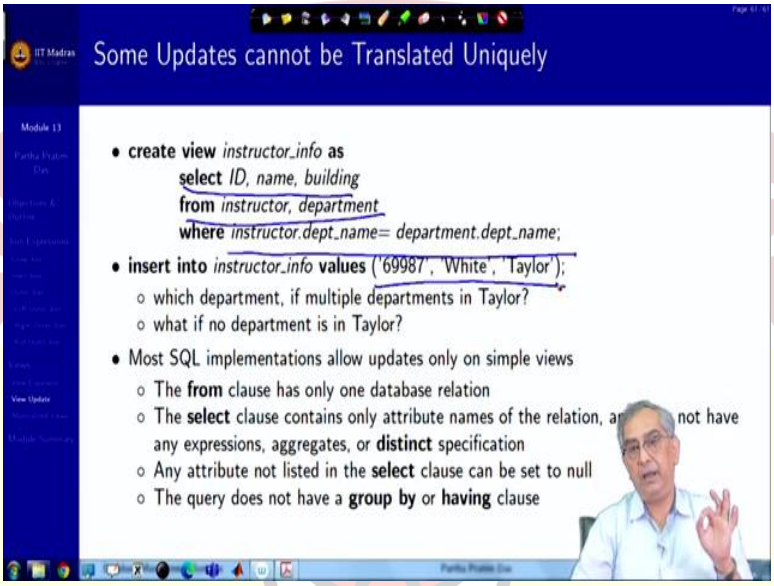
(Refer Slide Time: 27:51)



Now, the question is treating the view as a relation what happens if I do an update. So, suppose I do an insert what will happen? Naturally, if I do an insert, that insert to happen view is not physical. It is an expression, it is a query. So, if I do an insert in that the insert actually has to happen in the original physical relation. So, if I insert into faculty, this ID, name and department, then actual relation of instructor has a salary field.

So, this insert will be possible if that salary field is nullable. And in the instructor relation actually this quadruplet will get inserted. If that is possible, this update will happen if this is not possible, the update cannot take place.

(Refer Slide Time: 28:47)

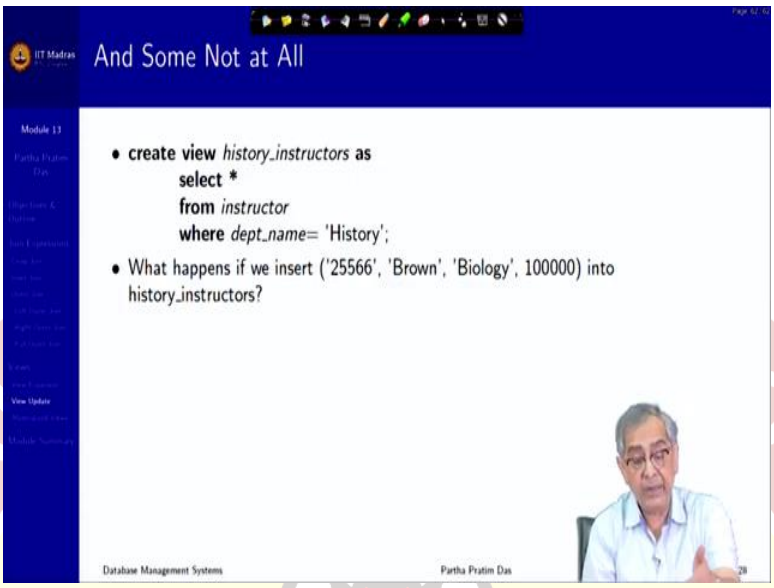


Now, there are problems with updating the views because there are issues which cannot be uniquely translated in terms of, so you are updating on the view and you want to translate in the physical relation but not always that can be uniquely done. Just think about this. ID, name, building, instructor, department, department name, instructor.

Now, we are inserting into this instructor info, you are instructing an ID, the name of the instructor and the building. Now, you cannot independently do that because the department name is not known. So, if Taylor building has multiple departments, which department name would you use? If there is no department in the Taylor building, what do you use, so there is not a clear unique semantics if you allow to do that.

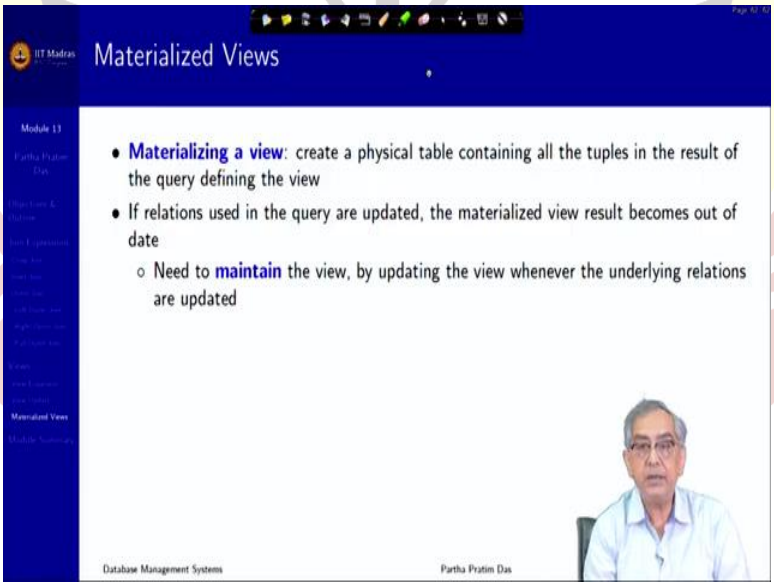
So, updates in terms of views often will be restricted it is best to keep it, I mean most SQL implementation allow updates only on single relation view, simple views, that is there is only one relation and select clauses only attributes from that relation and so on. Not very complicated things, just check up on your database manual to see what kind of view updates are allowed on your particular system. But it is not a good practice to try to update views very often, it is better to do that only occasionally.

(Refer Slide Time: 30:25)



Because there could be instances like this, where I have a view history instructor, which is where department name is history, and I am trying to insert a tuple where the department name is biology, this can never be done. Because if we allow this, then immediately the view get invalidated.

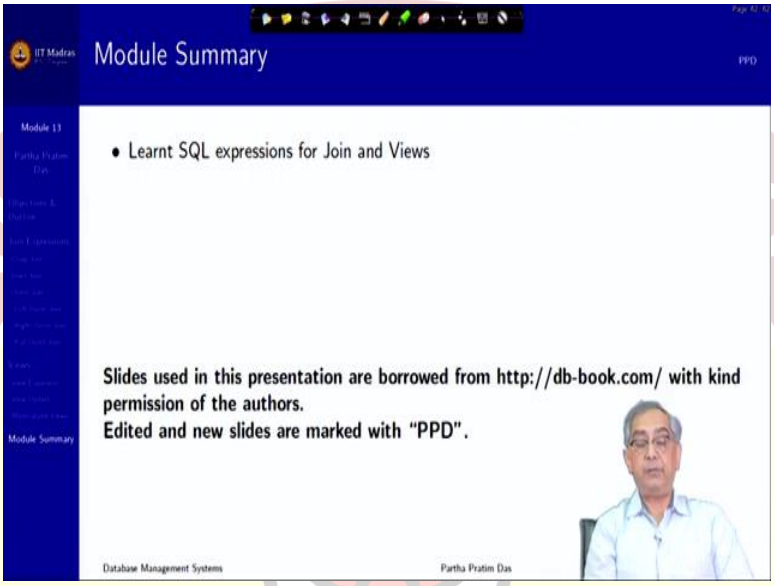
(Refer Slide Time: 30:47)



So, now the last point is, well, the view is virtual, but if you want at any point, you can materialize that view, that is whatever you are seeing in that virtual relation,` we can create a table, update it with the then tuples of the view, but if you do that, if you materialize, make it a

physical table, then naturally it is, it will become inconsistent with the future views of the view because the view will keep on changing but this has got. So, it will need an update if you have materialized view.

(Refer Slide Time: 31:25)



So, we have learnt about join expression and views in SQL today at the intermediate level. Thank you all very much for your attention. See you in the next module which will be the last module in the intermediate SQL discussions and we will discuss some more very, very interesting features of SQL.